

Traps

2026-03-23

1s2026

Gabriela Bittencourt, João Pedro Leôncio and Vinícius Peixoto

Contents



1	Introduction	6
1.a	Motivation	7
1.b	Idea: polling	8
2	Traps	11
2.a	Definition	12
2.b	Example	13
2.c	Trap numbers	14
2.d	Trap handlers	18
3	Understanding Resources	19
3.a	Memory x Storage	20
3.b	Program x Process	21
3.c	Memory Management Historical Advance	22
3.d	Non-contiguous Memory Allocation	23
4	Memory management overview	24
4.a	Physical vs. virtual address spaces	25
4.b	Some terminology	28

Contents (ii)



5	Physical memory allocation	29
5.a	Physical memory allocation	30
6	Virtual memory	32
6.a	Virtual memory	33
6.b	Page tables	34
6.c	Walking the page tables	37
7	Bitwise idioms	38
7.a	Basics	39
7.b	Bitmasks	42
8	Configuring virtual memory and paging	49
8.a	Common idioms	50
9	Lab: Memory management	52
9.a	files	53
10	Free Pages	55
10.a	Linked list	56
10.b	Data Structs options for the freelist	57

Contents (iii)



11	Allocating	58
11.a	Sections	59
11.b	Attention when implementing	60
12	References	61
12.a	References	62

Topics braindump:

- Motivation for traps
 - Example of reading from mouse/kb
 - Example of how to deal with invalid operation (e.g. page faults)
 - Arrive at the conclusion that we need some kind of preemption to handle these
- Definition
 - Traps = stop current execution and jump to another address upon an event
 - Synchronous: software traps (exceptions)
 - Example: page faults
 - Mention page swapping, CoW, etc

Contents (iv)



- ▶ Asynchronous: external traps (interrupts)
 - Example: timers (systick, rtc, etc), serial, etc
- Trap considerations
 - ▶ Anatomy of a trap handler
 - Interrupt enabling disabling in registers
 - ▶ Need handlers to be fast
 - Mention how Linux does this: top halves vs. bottom halves
 - ▶ Talk about the RISC-V ABI, trap handler needs to save caller-saved registers
 - ▶ All code in the kernel has to assume it can be **preempted**
- Introduction to concurrency
 - ▶ Concurrency vs. parallelism
 - Example: Python `async`
 - ▶ Basic race conditions
 - ▶ Need for locking mechanisms

1 Introduction

Motivation



Certain events in a system may need immediate attention:

- Devices
 - User input, such as typing on a keyboard, moving a mouse, etc.
 - GPU finishes a job (e.g. rendering the next frame of your videogame)
 - Network interface card (NIC) receives a new network packet
 - And many more
- Software error conditions (e.g. someone messed up):
 - Invalid instructions
 - Page faults (invalid access to pages)

Question: how to deal with those events?

Idea: polling



- Most hardware devices are exposed to software through **memory-mapped registers**
 - e.g. hardware registers can be accessed by reading/writing to memory addresses
 - For example, the serial device our kernel uses to print to the console:

```
char *uart = (char*)0x10000000;  
void putchar(char c)  
{  
    *uart = c;  
    return;  
}
```

- But also many others, e.g. PCI devices (GPUs, NICs, SATA/NVMe controllers, etc.)

Idea: polling (ii)



Idea: continuously check those memory-mapped registers for any events

- This is called **busy-waiting**

```
#define DEV_DATA_AVAIL  *(u64*)0x20000000;  
#define DEV_DATA        *(u64*)0x20000008;  
u64 dev_read()  
{  
    volatile u64 data_available = DEV_REG_DATA_AVAIL;  
    volatile u64 data = DEV_REG_DATA;  
  
    while (1) {  
        if (data_available != 0)  
            return data;  
    }  
}
```

What is the **problem** with this?

Idea: polling (iii)



What is the **problem** with this?

- We waste a lot of CPU cycles checking if data is available
- What if we need to do other work in the meantime?
- Stop doing work periodically to check?
 - How often should we check without risking missing data because we were too slow?
- This game gets complicated very quickly when you have multiple devices
- And what if we have a **bug** in our driver (e.g. nullptr deref)?
- Crash the whole system?

There is a better solution: **traps**

2 Traps

Definition



- A **trap** is an event that causes the CPU to stop whatever it is doing and perform a **jump** to another code location upon an event
- There are generally two types of events:
 - ▶ **External**: an external device raises an event that requires immediate attention
 - e.g. a character has arrived in our serial device, or a network packet arrived in a NIC, etc.
 - These are commonly called **interrupts**, or **IRQs** (= interrupt requests)
 - ▶ **Internal**: something in the code that the CPU is running raises an event that requires immediate attention
 - e.g. a page fault (accessing invalid pages), but also dividing by zero, executing privileged or invalid instructions, etc
 - These are commonly called **exceptions** or **faults**

How would our serial driver example work with interrupts?

Example



```
struct {
    // buffer for the data
    u64 data[512];
    // elements available to read
    size_t available;
} dev;

#define DEV_DATA *(u64*)0x20000008
void dev_irq()
{
    volatile u64 reg_data = DEV_DATA;
    dev.data[dev.end++] = reg_data;
    dev.data_available = true;
}
```

```
u64 dev_read()
{
    if (dev.available != 0)
        return dev.data[--dev.end];
    else
        return 0;
}
```

Note how we:

- `dev_irq` is called **asynchronously** to read data upon an interrupt
- We don't block when doing `dev_read` anymore
- We use a buffer → no more risk of dropping data

Trap numbers



There are several different kinds of traps. On most architectures, each trap type is identified by a **number**

- For exceptions, there are usually **architecturally defined** codes identifying each exception type
- For interrupts, there are usually **platform defined** numbers identifying each interrupt type
 - ▶ These are called **IRQ numbers** in the Linux world
 - ▶ Generally used for hardware interrupts
 - ▶ Usually correspond to physical hardware pins connected to the CPU/interrupt controller

Trap numbers (ii)

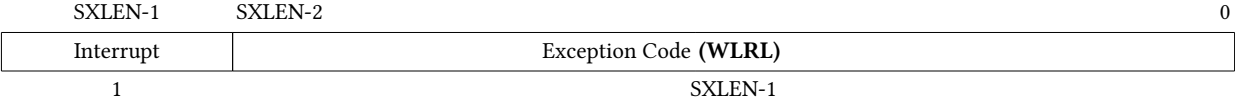


Figure 1: The `scause` CSR in RISC-V.

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2-4	Reserved
1	5	Supervisor timer interrupt
1	6-8	Reserved
1	9	Supervisor external interrupt
1	10-12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥ 16	Designated for platform use

Figure 2: The `scause` interrupt codes in RISC-V.

Trap numbers (iii)



0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10-11	<i>Reserved</i>
0	12	Instruction page fault
0	13	Load page fault
0	14	<i>Reserved</i>
0	15	Store/AMO page fault
0	16-17	<i>Reserved</i>
0	18	Software check
0	19	Hardware error
0	20-23	<i>Reserved</i>
0	24-31	<i>Designated for custom use</i>
0	32-47	<i>Reserved</i>
0	48-63	<i>Designated for custom use</i>
0	≥ 64	<i>Reserved</i>

Figure 3: The `scause` exception codes in RISC-V.

Trap numbers (iv)



Keep in mind the fundamental difference between **exceptions** and **interrupts**:

- Exceptions are **synchronous**:
 - ▶ Triggered upon executing a code instruction
 - Illegal instruction
 - Load/store/instruction page faults
 - Environment calls (`ecall`)
 - ▶ Identifying codes are architecturally **fixed**
- Interrupts are **asynchronous**:
 - ▶ Triggered by various hardware events that may happen at any time
 - Can have various meanings depending on hardware (serial device has data available, GPU finishes rendering frame, etc.)
 - ▶ IRQ numbers are defined by the **platform** (e.g. each defined by the architecture of the motherboard/system-on-chip)

Trap handlers



A **trap handler** is a function that the CPU jumps to when a trap happens.

There are normally two ways of going about it:

- Have a single function that handles all traps
 - This is what is generally used in RISC-V (AFAIK)
- Have an array of functions that handle different trap types (generally interrupts)
 - This scheme is what is usually called **vectored interrupt** (and the array is the **interrupt vector table**)
 - IRQ 0 triggered → CPU jumps to the address stored in `IVT[0]`, IRQ 1 → `IVT[1]` and so on

3 Understanding Resources

Memory x Storage



When we talk about ‘memory’ in OS context, we are talking about **RAM**

Memory is NOT Storage

- Memory
 - fast, expensive, volatile
- Storage
 - slow, cheap, consistent

Program x Process



- Program
 - Code. Static entity.
 - Needs storage
 - Example: Your C code from MC202: the .c file
- Process
 - The execution of the code. Dynamic entity.
 - Needs memory: the instructions will be load in specific space of memory (), variables will be set to specific spaces in memory ()
 - Example: The instance of your text editor while you are writing a program; the instance of the terminal running when you are navigating through it to compile your code and when you run it; the instance of your program during the short period of time your computer is executing your code

Memory Management Historical Advance



- Contiguous Memory Allocation (no abstraction of memory)
 - ▶ One part is reserved to the OS, the other runs a single process
 - ▶ One process at the time: in order to change the process executed, the OS must save the previous process in disk and then load the next one.
 - ▶ Simple memory management
 - ▶ Poor memory utilization
- Non-contiguous Memory Allocation
 - ▶ Allows each process to “see” the entire address space as its own
 - ▶ Multiple processes can coexist
 - ▶ Memory allocation logic is opaque to processes – the kernel handles virtual memory

Non-contiguous Memory Allocation

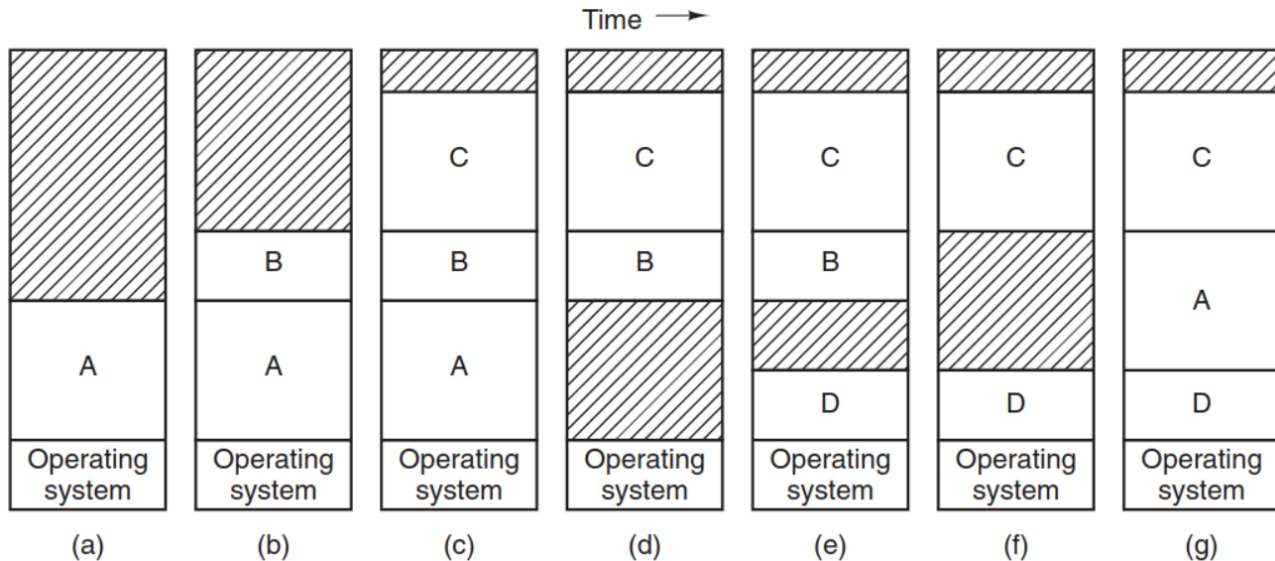


Figure 4: Memory allocation of 5 processes (programs: A, B, C, D and A again) over time.

4 Memory management overview

Physical vs. virtual address spaces



- **Physical addresses:** actual addresses as seen by the CPU on the system memory bus (e.g. DDR RAM, memory-mapped PCIe registers, etc.)
- **Virtual addresses:** made-up memory addresses that are mapped onto physical memory addresses

Example: `vmlinux` (a Linux kernel image)

```
$ llvm-objdump -t ~/kernel/linux/vmlinux
/home/nuke/kernel/linux/vmlinux:      file format elf64-littleaarch64
SYMBOL TABLE:
ffff80008228b738 l      .init.text      0000000000000054 record_mmu_state
ffff80008228b790 l      .init.text      0000000000000030 preserve_boot_args
ffff8000822655c8 l      F .rodata.text 0000000000000034 __primary_switch
ffff80008228b738 l      .init.text      0000000000000000 $x
ffff80008228b7c0 l      F .init.text      000000000000008c __primary_switched
ffff800080061230 l      F .text      0000000000000020 set_cpu_boot_mode_flag
```

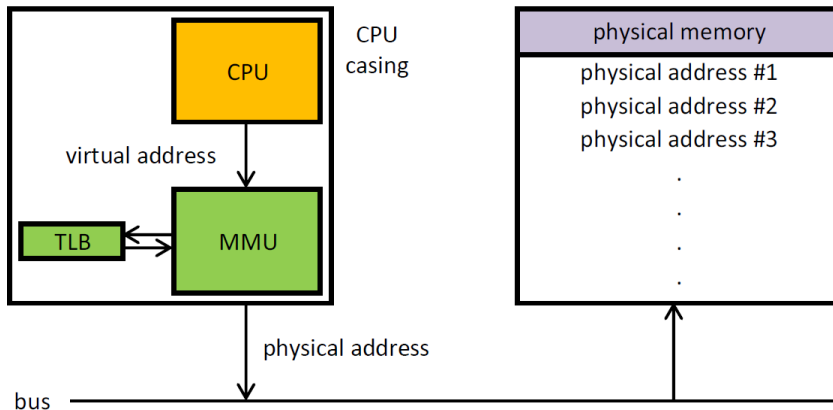
Note that the addresses for the symbols hover around address `0xffff8000822800` $\approx$ 65535 TiB (!)

Physical vs. virtual address spaces (ii)



- Clearly no system has that much memory
 - Might need to revisit this claim in the next decade or so :-)
- Those are **virtual addresses** configured by the kernel and translated into physical addresses
- This translation is done by a hardware unit called the Memory Management Unit (MMU)
 - The MMU is in turn configured by setting up data structures called **page tables**
 - Page tables describe the mapping between the virtual ↔ physical address spaces

Physical vs. virtual address spaces (iii)



CPU: Central Processing Unit

MMU: Memory Management Unit

TLB: Translation lookaside buffer

Figure 5: Overview of the address translation process.

Some terminology



- A **page** is a **block of memory** of fixed size
 - All memory is managed in units of pages
 - Traditionally 4KiB, more recently also 16KiB and 64KiB
 - In this class we always assume `PAGE_SIZE = 4096`
- Physical page number (**PPN**): the number of the n-th page in physical memory
 - Linux also calls this the **PFN** (page frame number)
 - Small exercise: what's the physical address of PPN=1024?
 - Another exercise: what's the PPN of physical address `0xffffffffabcde000`?
- Some commonly used idioms:
 - `PAGE_SHIFT`: number of bits to right-shift in order to get the PPN
 - For 4K pages, `PAGE_SHIFT = 12` ($2^{12} = 4096$)
 - `PAGE_SIZE`: the size of a page in bytes
 - `PAGE_SIZE = (1 << PAGE_SHIFT)`

5 Physical memory allocation

Physical memory allocation

Doing pretty much anything requires allocating physical memory. One of the first things a kernel needs to implement is a **physical page allocator**:

- Responsible for managing the available physical memory pages in the system
- Needs to keep track of which pages are in use and which ones are free
- Dumb (but perfectly valid) solution: freelist allocator
 - ▶ Linked list of all available pages
 - ▶ `alloc()` → pop the list head
 - ▶ `free(page)` → insert `page` at the front of the list

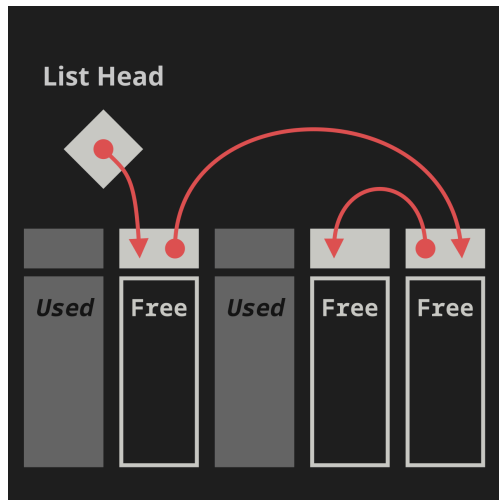


Figure 6: The freelist allocator (a linked-list of free pages)

Physical memory allocation (ii)



- This allocator has the upside of being dead simple to implement/debug
- But it would be nice to be able to allocate multiple contiguous pages at once
- In the next class we will go over other more elaborate allocators that solve this problem

6 Virtual memory

Virtual memory



- `riscv64` has 3 virtual memory modes (Sv39, Sv48 and Sv57)
 - We will work exclusively with Sv39 in this class
- We manage virtual memory by mapping **virtual page numbers** (VPNs) to **physical address numbers** (PPNs):

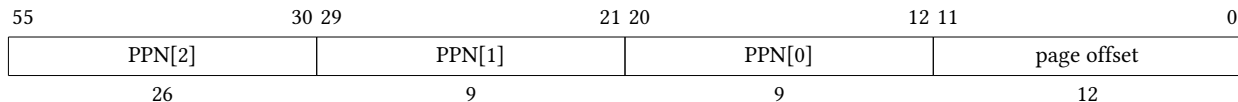


Figure 7: The Sv39 physical address encoding.

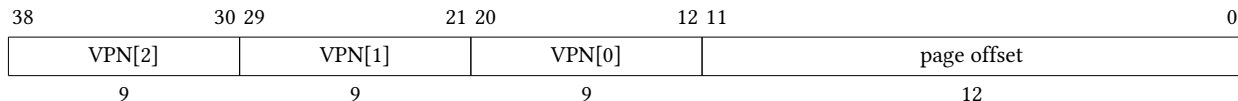


Figure 8: The Sv39 virtual address encoding.

Page tables



- In Sv39, a **page table** (PTB) is an array of $2^9 = 512$ words called **page table entries (PTEs)**
 - Note that in riscv64 1 word = 64 bits = 8 bytes and that $512 \times 8 \text{ bytes} = 4\text{KB}$
 - e.g. each page table takes up a physical page (not a coincidence)
- Each PTE can be either a:
 - **leaf page**: Map a VPN to a PPN
 - **non-leaf page**: Point to the PPN of another page table
- Sv39 uses a **3-level page table** scheme, which means that:
 - The **root page table** (also called a **level 1 page table**) contains entries that point to the PPN of a **level 2 page table**
 - Each **level 2 page table** points to a **level 3 page table** containing only leaf pages that map
- The process of following entries from root PTB $\rightarrow$ level 2 PTB $\rightarrow$ level 3 PTB until we reach a leaf page is called **walking** the page tables

Let's take a closer look at the structure of a PTE:

Page tables (ii)



63	62	61	60	54	53	28	27	19	18	10	9	8	7	6	5	4	3	2	1	0
N	PBMT	Reserved			PPN[2]		PPN[1]		PPN[0]		RSW		D	A	G	U	X	W	R	V
1	2	7			26		9		9		2		1	1	1	1	1	1	1	1

Figure 9: Sv39 page table entry.

The X/W/R bits have the following meaning:

X	W	R	meaning
0	0	0	Pointer to a next level page table
0	0	1	Read-only page
0	1	0	Reserved for future use
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
0	1	0	Reserved for future use
1	1	1	Read-write-execute page

Page tables (iii)

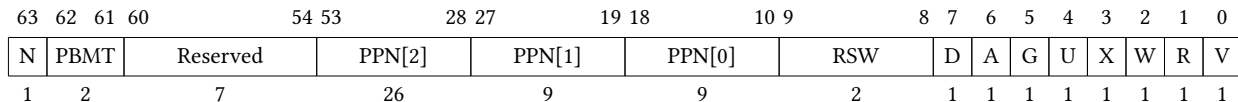
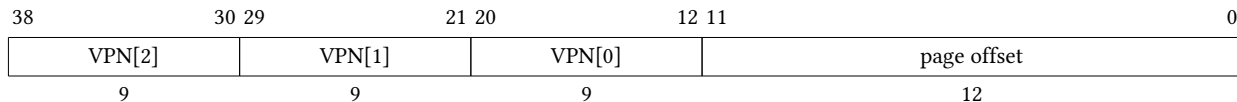


Figure 10: Sv39 page table entry.

For the others:

- **V** (valid): whether the PTE is valid
- **U** (user): whether the page is accessible in U-mode
- **G** (global): whether this is a global mapping that exists in all address spaces
- **A** (access): whether the page has been read/written/fetched since the bit was cleared
- **D** (dirty): whether the page has been written since the bit was cleared

Walking the page tables



We will go into more detail about how to actually implement this, but the (simplified) algorithm for walking the page tables is as follows:

- Start at the root page table (level 1)
- Check if the PTE at index `VPN[2]` has `V=1`; fail otherwise
- Move to the level 2 PTB at the physical address stored in `PPN[2:0]`
- Check if the PTE at index `VPN[1]` has `V=1`; fail otherwise
- Move to the level 1 PTB at the physical address stored in `PPN[2:0]`
- Extract the PPN that maps to the original VPN by looking at `PPN[2:0]`

7 Bitwise idioms

Basics



Bitwise operations: NOT ($\sim$), AND ($\&$), OR ($|$), XOR ($\wedge$)

A: 10110001

B: 01111001

$\sim$ A: 01001110

$\sim$ B: 10000110

A & B:

```
  10110001
& 01111001
-----
= 00110001
```

A | B:

```
  10110001
| 01111001
-----
= 00000110
```

A $\wedge$ B:

```
  10110001
^ 01111001
-----
= 00000110
```

Basics (ii)



Left shift: <<

```
A:      1000000010110001
A << 5: 0001011000100000
```

n bit left-shift $\rightarrow$ multiply by 2^n

Logical right shift: >>

```
A:      1000000010110001
A >> 5: 0000010000000101
```

n bit logical right-shift $\rightarrow$ (unsigned) divide by 2^n

Basics (iii)



Logical right shift: `>>`

```
A:      1000000010110001
A >> 5: 0000010000000101
```

n bit (logical) right-shift $\rightarrow$ (unsigned) divide by 2^n

Arithmetic right shift: `>>`

```
A:      1000000010110001
A >> 5: 1111110000000101
```

n bit (arithmetic) right-shift $\rightarrow$ (signed) divide by 2^n

Watch out: in C, unsigned $\rightarrow$ **logical** left-shift, signed $\rightarrow$ **arithmetic** left-shift

```
unsigned int A = 0b1000000010110001;
A >>= 5;      // 0b0000010000000101
```

```
int A =      0b1000000010110001;
A >>= 5;     // 0b1111110000000101
```

Bitmasks



A **bitmask** is a carefully constructed number that helps us **read/write specific bits** of another number.

- Quick terminology note: “**clear** a bit” means **make it 0**, “**set** a bit” means **make it 1**

Suppose we have have `A = 0b1000000010110001`, and want to **read bit 4**. We use the following trick:

Read `A[4]`: `A & (1 << 4)`

â

`A` `0b1000000010110001`

`MASK` `& 0b00000000000010000` ---> `(1 << 4)`

`= 0b00000000000010000`

Bitmasks (ii)



Similarly, if we wanted to **set bit 6**:

Set A[6]: $A \mid (1 \ll 6)$:

$\hat{A}$

A 0b1000000010110001

MASK | 0b0000000001000000 ---> $(1 \ll 6)$

= 0b1000000011110001

Bitmasks (iii)



What about **clearing bit 5**?

Clear A[5]: $A \& \sim(1 \ll 5)$:

$\hat{A}$

A 0b1000000010110001

~MASK & 0b1111111110111111 ---> $\sim(1 \ll 5)$

= 0b1000000010010001

Bitmasks (iv)



Notice the pattern:

- We have some number (or register) `A`
- Want to modify bit `n`? $\rightarrow \text{MASK} = (1 \ll n)$
- Set bit `n` $\rightarrow A \mid= \text{MASK}$
- Clear bit `n` $\rightarrow A \&= \sim\text{MASK}$

Now let's generalize this to **multiple bits**. How to **set bits 12, 3 and 2**?

Bitmasks (v)



Set A[12], A[3:2]: A |= MASK

MASK = ((1 << 12) | (1 << 3) | (1 << 2))

â€œâ€œâ€œ

A 0b1000000010110001

MASK | 0b0001000000001100

= 0b1001000010111101

What about **clearing bits 15, 4 and 5**?

Bitmasks (vi)



```
Clear A[15], A[5:4]: A &= ~MASK
MASK = ((1 << 15) | (1 << 5) | (1 << 4))
      ^          ^ ^
A      0b10000000010110001
MASK   & 0b0111111111001111
      -----
      = 0b00000000010000001
```

Last example (may be relevant for your next lab ð): how to write 0b1101 to **bits 4, 5, 6 and 7**?

Bitmasks (vii)



At this point we might as well do it in plain C:

```
#define MASK ((1UL << 7) | (1UL << 6) | (1UL << 5) | (1UL << 4))
#define SHIFT 4
uint64_t a = 0b1000000010110001
a &= ~MASK; // clear a[7:4]
a |= (0b1101UL << SHIFT)
```

Notice we use `1UL` for the constants in the macro; numeric constants in C **default to `int` (signed)**, but we generally want to **avoid signed bit arithmetic**; the `UL` suffix tells specifies that the numeric constant is an **unsigned long int** (e.g. 64-bit unsigned integer).

Always try to avoid mixing signed and unsigned numbers as it is one of the biggest sources of **undefined behavior** in C.

8 Configuring virtual memory and paging

Common idioms

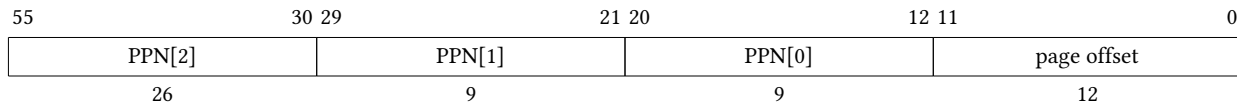


Figure 12: The Sv39 physical address encoding.

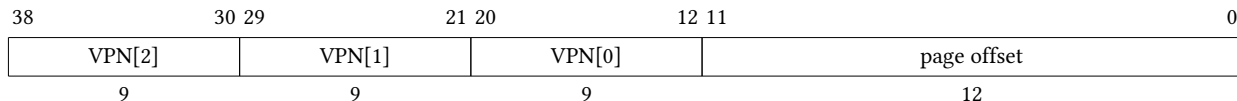


Figure 13: The Sv39 virtual address encoding.

```
#define PAGE_SHIFT 12
#define PAGE_SIZE (1 << PAGE_SHIFT)
```

Exercise: how to calculate `PAGE_MASK`?

Common idioms (ii)



Answer:

```
#define PAGE_MASK (PAGE_SIZE - 1)
```

9 Lab: Memory management

files



```
├─ include (kernel headers)
│   └─ arch
│       ├── pgtable.h (defs and helpers for page table)
│       └─ kalloc.h (freelist implementation)
└─ src
    └─ mm.c (implementation for memory management)
```

files (ii)



- `pgtable.h`: All the hardware specs and design for the page table
- `kalloc.h`: Implement a `freelist` – non-allocated pages of memory
 - `page_alloc()`: pop the data struct of freelist
 - `page_free()`: insert in data struct of freelist
- `mm.c`: Memory management
 - `ptb_walker`: walk through the levels of pagetable and returns when valid or !valid, or leaf or !leaf (it is up to implementation)
 - `vm_page_alloc`: allocate a page given a virtual address
 - `mm_init`: allocate all sessions: `.bss`, `.text`, `.data`, `.rodata`

10 Free Pages

Linked list



```
struct node {  
    int data;  
    struct node *next;  
}
```

To add new node:

```
struct create_node {  
    struct node new_node = (struct node*)malloc(sizeof(struct node)); // :(  
    // ...  
}
```

We cannot use a `malloc` when implementing the precursor of `malloc`.

We cannot dynamic alloc the memory if there is no memory management yet.

Data Structs options for the freelist



- Linked list, with all the nodes declared in array
- Directly an array with the addresses and a walker
- Bitmap the used pages and calculate the free addresses from the position of the bit
- Use the pages as node of the linked list – write at the first word, the address for next page

11 Allocating

Sections



Which flag to use in each section of the kernel executable? (PTE_READ , PTE_WRITE , PTE_EXEC)

- `.text`: contains executable instructions
- `.bss`: contains non-initialized statically allocated variables
- `.rodata`: contains read-only data
- `.data`: contains data that can be altered

Attention when implementing



```
*** Fatal Exception *** core dump ***  
Segmentation fault (core dumped)
```

Will never occur in your code.

12 References

References



- Tanenbaum, Andrew. (2012) '3. Memory Management', in *Modern operating systems*. Pearson Education, Inc., 4th ed.
- *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. <https://riscv.github.io/riscv-is-manual/snapshot/privileged/>
- *Virtual Memory Layout on RISC-V Linux*. <https://docs.kernel.org/arch/riscv/vm-layout.html>